1

Cyclohexane, cryptography, codes, and computer algebra

Three examples in this chapter illustrate some applications of the ideas and methods of computer algebra: the spatial configurations (conformations) of the cyclohexane molecule, a chemical problem with an intriguing geometric solution; a cryptographic protocol for the secure transmission of messages; and distributed codes for sharing secrets or sending packets over a faulty network. Throughout this book you will find such sample applications in a wide variety of areas, from the design of calendars and musical scales to image compression and the intersection of algebraic curves. The last section in this chapter gives a concise overview of some computer algebra systems.

1.1. Cyclohexane conformations

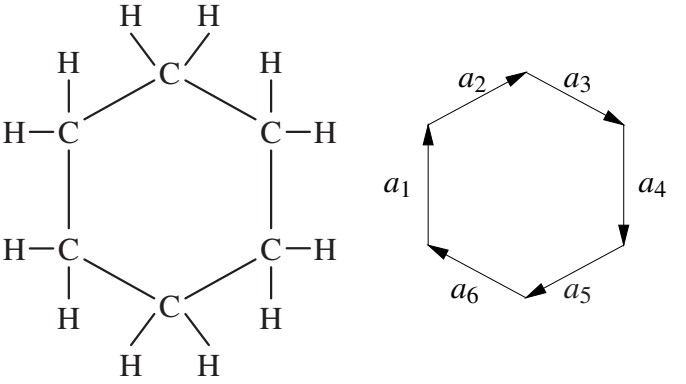


FIGURE 1.1: The structure formula for cyclohexane (C_6H_{12}), and the orientation we give to the bonds $a_1, \ldots, a_6$.

We start with an example from chemistry. It illustrates the three typical steps in mathematical applications: creating a mathematical model of the problem at hand, "solving" the model, and interpreting the solution in the original problem. Usually,

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

none of these steps is straightforward, and one often has to go back and modify the approach.

Cyclohexane C_6H_{12} (Figure 1.1), a molecule from organic chemistry, is a hydrocarbon consisting of six carbon atoms (C) connected to each other in a cycle and twelve hydrogen atoms (H), two attached to each carbon atom. The four bonds of one carbon atom (two bonds to adjacent carbon atoms and two bonds to hydrogen atoms) are arranged in the form of a tetrahedron, with the carbon in the center and its bonds pointing to the four corners. The angle α between any two bonds is about 109 degrees (the precise value of α satisfies $\cos\alpha = -1/3$). Two adjacent carbon atoms may freely rotate around the bond between them.

Chemists have observed that cyclohexane occurs in two incongruent conformations (which are not transformable into each other by rotations and reflections), a "chair" (Figure 1.2) and a "boat" (Figure 1.3), and experiments have shown that the "chair" occurs far more frequently than the "boat". The frequency of occurrence of a conformation depends on its free energy—a general rule is that molecules try to minimize the free energy—which in turn depends on the spatial structure.

When modeling the molecule by means of plastic tubes (Figure 1.4) representing the carbon atoms and the bonds between them (omitting the hydrogen atoms for simplicity) in such a way that rotations around the bonds are possible, one observes that there is a certain amount of freedom in moving the atoms by rotations around the bonds in the "boat" conformation (we will call it the **flexible** conformation), but that the "chair" conformation is **rigid**, and that it appears to be impossible to get from the "boat" to the "chair" conformation. Can we mathematically model and, if possible, explicitly describe this behavior?

We let $a_1, \ldots, a_6 \in \mathbb{R}^3$ be the orientations of the six bonds in three-space, so that all six vectors point in the same direction around the cyclic structure (Figure 1.1), and normalize the distance between two adjacent carbon atoms to be one. By $u \star v = u_1v_1 + u_2v_2 + u_3v_3$ we denote the usual inner product of two vectors $u = (u_1, u_2, u_3)$ and $v = (v_1, v_2, v_3)$ in $\mathbb{R}^3$. The cosine theorem says that $u \star v = \|u\|_2 \cdot \|v\|_2 \cdot \cos\beta$, where $\|u\|_2 = (u \star u)^{1/2}$ is the Euclidean norm and $\beta \in [0, \pi]$ is the angle between u and v, when both vectors are rooted at the origin. The above conditions then lead to the following system of equations:

$$\begin{aligned} a_1 \star a_1 = a_2 \star a_2 = \cdots = a_6 \star a_6 &= 1, \\ a_1 \star a_2 = a_2 \star a_3 = \cdots = a_6 \star a_1 &= \frac{1}{3}, \\ a_1 + a_2 + \cdots + a_6 &= 0. \end{aligned} \tag{1}$$

The first line says that the length of each bond is 1. The second line expresses the fact that the angle between two bonds adjacent to the same carbon atom is α (the cosine is $1/3$ instead of $-1/3$ since, seen from the carbon atom, the two bonds

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

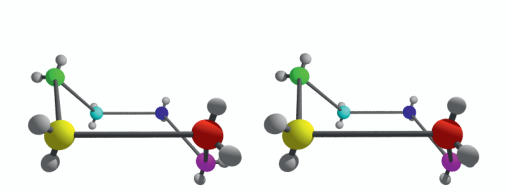


FIGURE 1.2: A stereo image of a "chair" conformation of cyclohexane. To see a three-dimensional image, hold the two pictures right in front of your eyes. Then relax your eyes and do not focus at the foreground, so that the two pictures fade away and each of them splits into two separate images (one for each eye). By further relaxing your eyes, try to match the right image that your left eye sees with the left image that your right eye sees. Now you see three images, each of the two outer ones only with one of your eyes, and the middle one with both eyes. Cautiously focus on the middle image, at the same time slowly moving the book away from your head, until the image is sharp. (This works best without wearing glasses.)

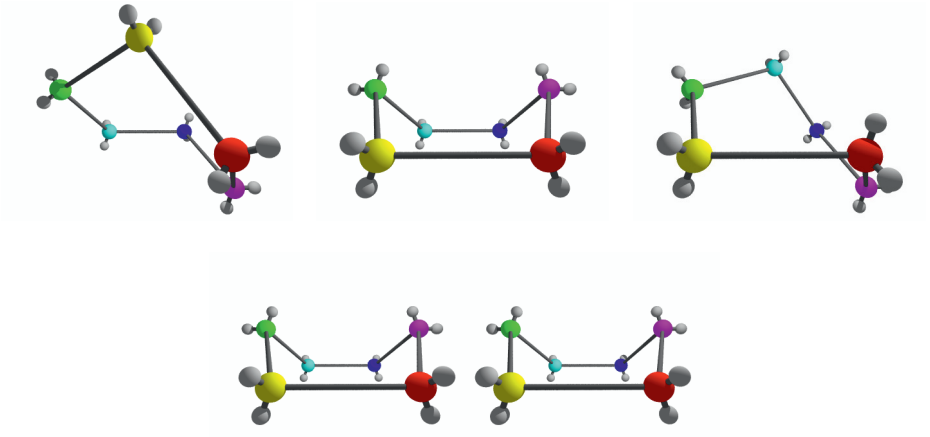


FIGURE 1.3: Three "boat" conformations of cyclohexane and a stereo image of the middle one (see Figure 1.2 for a viewing instruction).

have opposite orientation). Finally, the last line expresses the cyclic nature of the structure.

Together, (1) comprises $6+6+1=13$ equations in the 18 coordinates of the points $a_1,\ldots,a_6$. The first ones are quadratic, and the last ones are linear. There is still redundancy coming from the whole structure's possibility to move and rotate around freely in three-space. One possibility to remedy this is to introduce three more equations expressing the fact that a_1 and a_2 are parallel to the x-axis respectively the x,y-plane. These equations can be solved with a computer algebra system, but the resulting description of the solutions is highly complicated and non-intuitive.

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

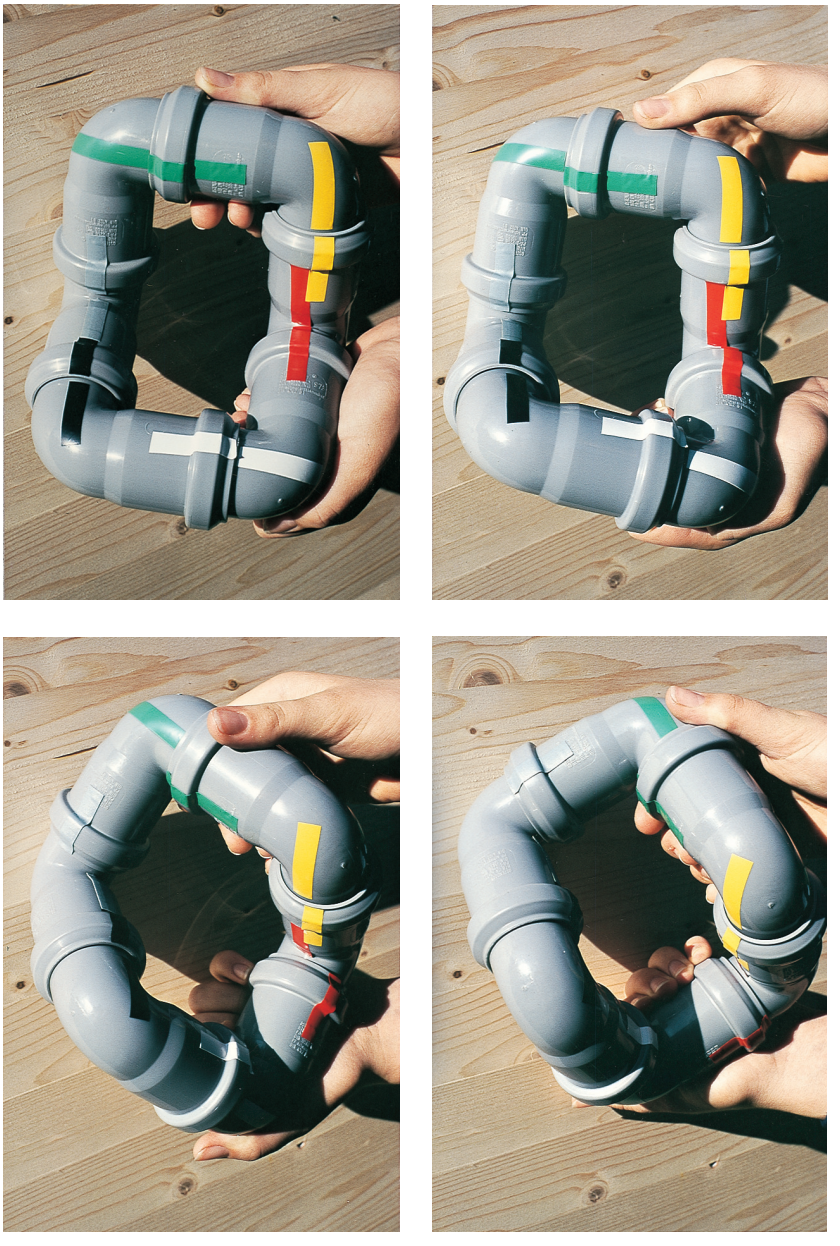


FIGURE 1.4: A plumbing knee model of cyclohexane, with nearly right angles.

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

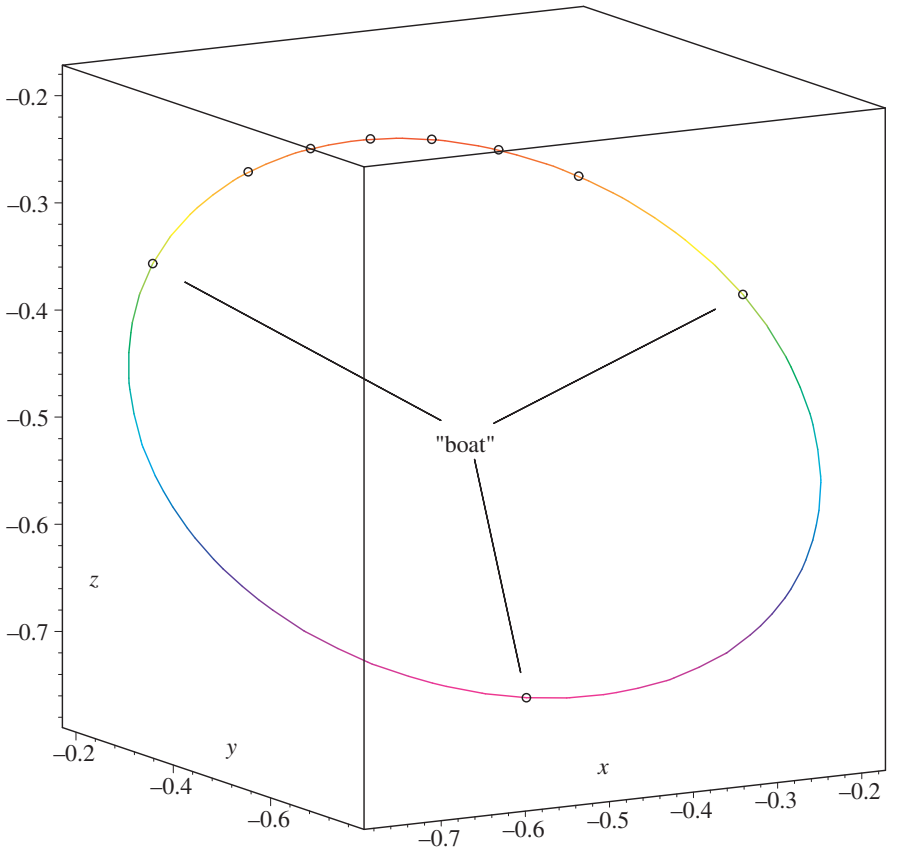


FIGURE 1.5: The curve E.

For a successful solution, we pursue a different, more symmetric approach, by taking the inner products $S_{ij} = a_i \star a_j$ for $1 \leq i, j \leq 6$ as unknowns instead of the coordinates of $a_1, \ldots, a_6$. This is described in detail in Section 24.4. Under the conditions (1), S_{ij} is the cosine of the angle between a_i and a_j. It turns out that all S_{ij} depend linearly on S_{13}, S_{35}, and S_{51}, and that the triples of values $(S_{13}, S_{35}, S_{51}) \in \mathbb{R}^3$ leading to the flexible conformations are given by the space curve E in Figure 1.5. The solution makes heavy use of various computer algebra tools, such as **resultants** (Chapter 6), **polynomial factorization** (Part III), and **Gröbner bases** (Chapter 21). The three marked points $(-1/3, -1/3, -7/9)$, $(-1/3, -7/9, -1/3)$ and $(-7/9, -1/3, -1/3)$ correspond to the three "boat" conformations in Figure 1.3 (all of them are equivalent by cyclically permuting $a_1, \ldots, a_6$). Actually, some information gets lost in the transition from the a_i to the S_{ij}, and each point on the curve E corresponds to precisely two spatial conformations which are mirror images of each other. The rigid conformation corresponds to the isolated solution $S_{13} = S_{35} = S_{51} = -1/3$ not lying on the curve.

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

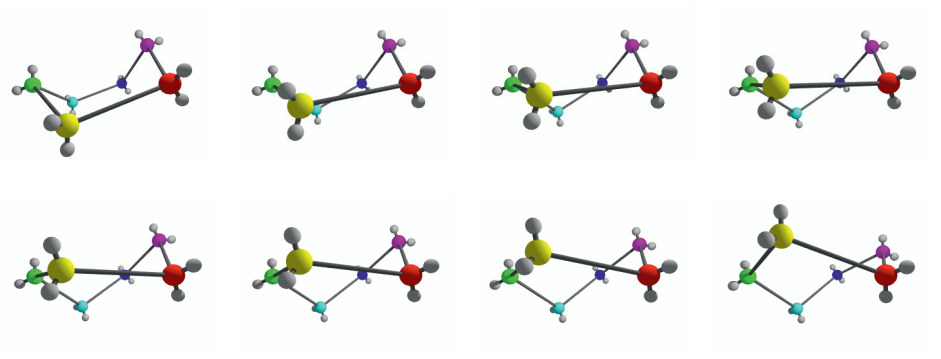
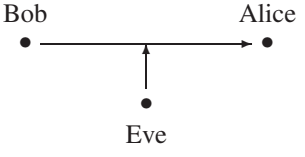


FIGURE 1.6: Eight flexible conformations of cyclohexane corresponding to the eight points marked in Figure 1.5. The first and the eighth ones are "boats". The point of view is such that the positions of the red, green, and blue carbon atoms are invariant for all eight pictures.

We built the simple physical "model" in Figure 1.4 of something similar to cyclohexane as follows. We bought six plastic plumbing "knees", with approximately a right angle. (German plumbing knees actually have an angle of about 93 degrees, for some deep hydrodynamic reason.) This differs considerably from the 109 degrees of the carbon tetrahedron, but on the other hand, it only cost about € 7. We stuck the six parts together and pulled an elastic cord through them to keep them from falling apart. Then one can smoothly turn the structure through the flexible conformations corresponding to the curve in Figure 1.5, physically "feeling" the curve. Pulling the whole thing forcibly apart, one can also get into the "chair" position. Now no wiggling or gentle twisting will move the structure; it is quite rigid.

1.2. The RSA cryptosystem

The basic scenario for cryptography is as follows. Bob wants to send a message to Alice in such a way that an eavesdropper (Eve) listening to the transmission channel cannot understand the message. This is done by enciphering the message so that only Alice, possessing the right **key**, can decipher it, but Eve, having no access to the key, has no chance to recover the message.



In classical **symmetric** cryptosystems, Alice and Bob use the same key for both encryption and decryption. The **RSA cryptosystem**, described in detail in

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

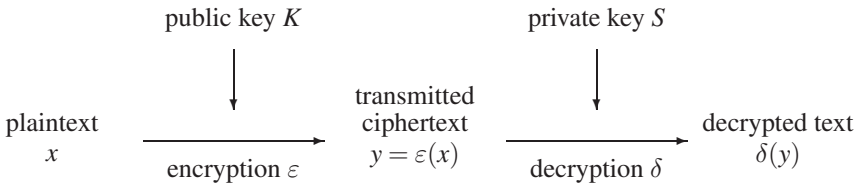


FIGURE 1.7: A public key cryptosystem.

Section 20.2, is an **asymmetric** or **public key cryptosystem**. Alice has a **public key** K that she publishes in some directory, and a **private key** S that she keeps secret. To encrypt a message, Bob uses Alice's public key, and only Alice can decrypt the ciphertext using her private key (Figure 1.7).

The RSA system works as follows. First, Alice randomly chooses two large (150 digit, say) prime numbers $p \neq q$ and computes their product $N = pq$. Efficient probabilistic **primality tests** (Chapter 18) make it easy to find such primes and then N, but the problem of finding p and q, when just N is given (that is, factoring N), seems very hard (see Chapter 19). Then she randomly chooses another integer $e \in \{2, \ldots, \varphi(N) - 2\}$ coprime to $\varphi(N)$, where φ is **Euler's totient function** (Section 4.2). For our particular N, the **Chinese Remainder Theorem** 5.3 implies that $\varphi(N) = (p-1) \cdot (q-1)$. Then Alice publishes the pair $K = (N, e)$. To obtain her private key, Alice uses the **Extended Euclidean Algorithm** 3.14 to compute $d \in \{2, \ldots, \varphi(N) - 2\}$ such that $ed \equiv 1 \bmod \varphi(N)$, and $S = (N, d)$ is her private key. Thus $(ed-1)/\varphi(N)$ is an integer.

Before they can exchange messages, both parties agree upon a way of encoding messages (pieces of text) as integers in the range $0, \ldots, N-1$ (this is not part of the cryptosystem itself). For example, if messages are built from the 26 letters A through Z, we might identify A with 0, B with 1, ..., Z with 25, and use the 26-ary representation for encoding. The message "CAESAR" is then encoded as

$$2 \cdot 26^0 + 0 \cdot 26^1 + 4 \cdot 26^2 + 18 \cdot 26^3 + 0 \cdot 26^4 + 17 \cdot 26^5 = 202\,302\,466.$$

Long messages are broken into pieces. Now Bob wants to send a message $x \in \{0, \ldots, N-1\}$ to Alice that only she can read. He looks up her public key (N, e), computes the encryption $y = \varepsilon(x) \in \{0, \ldots, N-1\}$ of x such that $y \equiv x^e \bmod N$, and sends y. Computing y can be done very efficiently using **repeated squaring** (Algorithm 4.8). To decrypt y, Alice uses her private key (N, d) to compute the decryption $x^* = \delta(y) \in \{0, \ldots, N-1\}$ of y with $x^* \equiv y^d \bmod N$. Now **Euler's theorem** (Section 18.1) says that $x^{\varphi(N)} \equiv 1 \bmod N$, if x and N are coprime. Thus

$$x^* \equiv y^d \equiv x^{ed} = x \cdot \left(x^{\varphi(N)}\right)^{(ed-1)/\varphi(N)} \equiv x \bmod N,$$

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

and it follows that $x^* = x$ since x and x^* are both in $\{0, \ldots, N-1\}$. In fact, $x^* = x$ also holds when x and N have a nontrivial common divisor.

Without knowledge of d, however, it seems currently infeasible to compute x from N, e, and y. The only known way to do this is to factor N into its prime factors, and then to compute d with the Extended Euclidean Algorithm as Alice did, but **factoring integers** (Chapter 19) is extremely time-consuming: 300 digit numbers are beyond the capabilities of currently known factoring algorithms even on modern supercomputers or workstation networks.

Software packages like PGP ("Pretty Good Privacy"; see Zimmermann (1996) and `http://www.openpgp.org`) use the RSA cryptosystem for encrypting and authenticating e-mail and data files, and for secure communication over local area networks or the internet.

1.3. Distributed data structures

We start with another problem from cryptography: **secret sharing**. Suppose that, for some positive integer n, we have n players that want to share a common secret in such a way that all of them together can reconstruct the secret but any subset of $n-1$ of them or less cannot do so. The reader may imagine that the secret is a key in a cryptosystem or a code guarding a common bank account or inheritance, or an authorization for a financial transaction of a company which requires the signature of a certain number of managers. This can be solved by using interpolation (Section 5.2), as follows.

We choose $2n-1$ values $f_1, \ldots, f_{n-1}, u_0, \ldots, u_{n-1}$ in a field F (say, F is $\mathbb{Q}$ or a **finite field**) such that the u_i are distinct, and let f be the polynomial $f_{n-1}x^{n-1} + \cdots + f_1 x + f_0$, where $f_0 \in F$ is the secret, encoded in an appropriate way. Then we give $v_i = f(u_i) = f_{n-1}u_i^{n-1} + \cdots + f_1 u_i + f_0$ to player i. The reconstruction of the polynomial f from its values $v_0, \ldots, v_{n-1}$ at the n distinct points $u_0, \ldots, u_{n-1}$ is called **interpolation** and may be performed, for example, by using the **Lagrange interpolation formula** (Section 5.2). The interpolating polynomial at n points of degree less than n is unique, and hence all n players together can recover f and the secret f_0, but one can show that any proper subset of them can obtain no information on the secret. More precisely, all elements of F—as potential values of f_0—are equally consistent with the knowledge of fewer than n players. We discuss the secret sharing scheme in Section 5.3.

Essentially the same scheme works for a different problem: reliable routing. Suppose that we want to send a message consisting of several packets over a network (for example, the internet) that occasionally loses packets. We want to encode a message of length n into not many more than n packets in such a way that after a loss of up to l arbitrary packets the message can still be recovered. Such a scheme is called an **erasure code**. (We briefly discuss the related **error correcting codes**, which are designed for networks that sometimes perturb packets but do not

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

lose them, in Chapter 7.) An obvious solution would be to send the message $l+1$ times, but this increases message length and hence slows down communication speed by a factor of $l+1$ and is unacceptable even for small values of l.

Again we may assume that each packet is encoded as an element of some field F, and that the whole message is the sequence of packets $f_0, \ldots, f_{n-1}$. Then we choose $k = n+l$ distinct evaluation points $u_0, \ldots, u_{k-1} \in F$ and send the k packets $f(u_0), \ldots, f(u_{k-1})$ over the net. Assuming that the sequence number i is contained in the packet header and that the recipient knows $u_0, \ldots, u_{k-1}$, she can reconstruct the original message—the (coefficients of the) polynomial f—from any n of the surviving packages by interpolation (and may discard any others).

The above scheme can also be used to distribute n data blocks (for example, records of a database) among $k = n+l$ computers in such a way that after failure of up to l of them the complete information can still be recovered. The difference between secret sharing and this scheme is that in the former the relevant piece of information is only *one* coefficient of f, while in the latter it is the whole polynomial.

The above methods can be viewed as problems in **distributed data structures**. Parallel and distributed computing is an active area of research in computer science. Developing algorithms and data structures for parallel computing is a nontrivial task, often more challenging than for sequential computing. The amount of parallelism that a particular problem admits is sometimes difficult to detect. In computer algebra, **modular algorithms** (Chapters 4 and 5) provide a "natural" parallelism for a certain class of algebraic problems. These are divided into smaller problems by reduction modulo several "primes", the subproblems can be solved independently in parallel, and the solution is put together using the **Chinese Remainder Algorithm** 5.4. An important particular case is when the "primes" are linear polynomials $x - u_i$. Then modular reduction corresponds to evaluation at u_i, and the Chinese Remainder Algorithm is just interpolation at all points u_i, as in the examples above.

If the interpolation points are **roots of unity** (Section 8.2), then there is a particularly efficient method for evaluating and interpolating at those points, the **Fast Fourier Transform** (Chapters 8 and 13). It is the starting point for efficient algorithms for polynomial (and integer) arithmetic in Part II.

1.4. Computer algebra systems

We give a short overview of the computer algebra systems available at the time of writing. We do not present much detail, nor do we aim for completeness.

The vast majority of this book's material is of a fundamental nature and technology-independent. But this short section and some other places where we discuss implementations date the book; if the rapid progress in this area continues as expected, some of this material will become obsolete in a short time.

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

Computer algebra systems have historically evolved in several stages. An early forerunner was Williams' (1961) PMS, which could calculate floating point polynomial gcd's. The *first generation*, beginning in the late 1960s, comprised MACSYMA from Joel Moses's MATHLAB group at MIT, SCRATCHPAD from Richard Jenks at IBM, REDUCE by Tony Hearn, and SAC-I (now SACLIB) by George Collins. MUMATH by David Stoutemyer ran on a small microprocessor; its successor DERIVE is available on the hand-held TI-92. These researchers and their teams developed systems with algebraic engines capable of doing amazing *exact* (or *formal* or *symbolic*) computations: differentiation, integration, factorization, etc. The *second generation* started with MAPLE by Keith Geddes and Gaston Gonnet from the University of Waterloo in 1985 and MATHEMATICA by Stephen Wolfram. They began to provide modern interfaces and graphic capabilities, and the hype surrounding the launching of MATHEMATICA did much to make these systems widely known. The *third generation* is on the market now: AXIOM, a successor of SCRATCHPAD, by NAG, MAGMA by John Cannon at the University of Sydney, and MUPAD by Benno Fuchssteiner at the University of Paderborn. These systems incorporate a categorical approach and operator calculations.

Today's research and development of computer algebra systems is driven by three goals, which sometimes conflict: wide functionality (the capability of solving a large range of different problems), ease of use (user interface, graphics display), and speed (how big a problem you can solve with a routine calculation, say in a day on a workstation). This text will concentrate on the latter goal. We will see the basics of the fastest algorithms available today, mainly for some problems in polynomial manipulation. Several groups have developed software for these basic operations: PARI by Henri Cohen in Bordeaux, LIDIA by Johannes Buchmann in Darmstadt, NTL by Victor Shoup, and packages by Erich Kaltofen. William Stein's open-source SAGE facilitates interoperability of various systems, and SINGULAR is strong in certain algebraic settings.

A central problem is the factorization of polynomials. Enormous progress has been achieved, in particular over finite fields. In 1991, the largest problems amenable to routine calculation were about 2 KB in size, while in 1995 Shoup's software could handle problems of about 500 KB. Almost all the progress here is due to new algorithmic ideas, and this problem will serve as a guiding light for this text.

Computer algebra systems have a wide variety of applications in fields that require computations that are tedious, lengthy and difficult to get right when done by hand. In physics, computer algebra systems are used in high energy physics, for quantum electrodynamics, quantum chromodynamics, satellite orbit and rocket trajectory computations and celestial mechanics in general. As an example, Delaunay calculated the orbit of the moon under the influence of the sun and a nonspherical earth with a tilted ecliptic. This work took twenty years to complete and was published in 1867. It was shown, in 20 hours on a small computer in 1970, to be correct to nine decimal places.

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

Important implementation issues for a general-purpose computer algebra system concern things like the user interface (see Kajler & Soiffer 1998 for an overview), memory management and garbage collection, and which representations and simplifications are allowed for the various algebraic objects.

Their power of visualization and of solving nontrivial examples makes computer algebra systems more and more appealing for use in education. Many topics in calculus and linear algebra can be beautifully illustrated with this technology. The eleven reports in the Special Issue of the *Journal of Symbolic Computation* (Lambe 1997) give an idea of what can be achieved. The (self-referential) use in computer algebra courses is obvious; in fact, this text grew out of courses that the first author has taught at several institutions, using MAPLE (and later other systems) since 1986.

As in most active branches of science, the *sociology* of computer algebra is shaped by its leading conferences, journals, and the researchers running these. The premier research conference is the annual International Symposium on Symbolic and Algebraic Computation (ISSAC), into which several earlier streams of meetings have been amalgamated. It is run by a Steering Committee, and national societies such as the US ACM's *Special Interest Group on Symbolic and Algebraic Manipulation* (SIGSAM), the *Fachgruppe Computeralgebra* of the German GI, and others, are heavily involved. In addition, there are specialized meetings such as MEGA (multivariate polynomials), DISCO (distributed systems), and PASCO (parallel computation). Sometimes relevant results are presented at related conferences, such as STOC, FOCS, or AAECC. Several computer algebra systems vendors organize regular workshops or user group meetings.

The highly successful *Journal of Symbolic Computation*, created in 1985 by Bruno Buchberger, is the undisputed leader for research publications. Some high-quality journals with a different focus sometimes contain articles in the field: *computational complexity*, *Journal of the ACM*, *Mathematics of Computation*, and *SIAM Journal on Computing*. Other journals, such as *AAECC* and *Theoretical Computer Science*, have some overlap with computer algebra.

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.002 Published online by Cambridge University Press